

Company Engineering Handbook

Version 1.0

1. Introduction

This handbook defines the engineering standards for all software projects developed by the company. Every project should follow these standards to ensure consistency, maintainability, scalability, security, and high code quality.

2. Engineering Philosophy

Our engineering philosophy is built around the following principles:

- Simplicity over complexity
 - Reusable code over duplicated code
 - Clean Architecture
 - Feature-Based Development
 - Security by default
 - Mobile-first development
 - SEO-first web development
 - Scalability from day one
 - Consistent engineering standards
 - Continuous improvement
-

3. Technology Stack

Frontend

Languages

- HTML5
- CSS3
- TypeScript

Frameworks & Libraries

- Next.js
- React

- Tailwind CSS
 - shadcn/ui
 - TanStack Query
 - React Hook Form
 - Zod
 - Axios
 - Lucide React
 - Recharts
 - Framer Motion
 - Sonner
-

Backend

Language

- TypeScript

Runtime

- Node.js

Framework

- NestJS

ORM

- Prisma
-

Database

- PostgreSQL
-

Development Tools

- Git
- GitHub
- Docker
- Docker Compose
- Swagger (OpenAPI)
- Postman
- ESLint

- Prettier

4. Software Architecture

Backend

- Clean Architecture
- Modular Monolith
- Feature-Based Organization
- Repository Pattern
- Use Case Pattern
- Dependency Injection

Backend Request Flow:

Client



Controller (Presentation)



Use Case (Application)



Repository Interface (Domain)



Repository Implementation (Infrastructure)



Prisma



PostgreSQL

Frontend

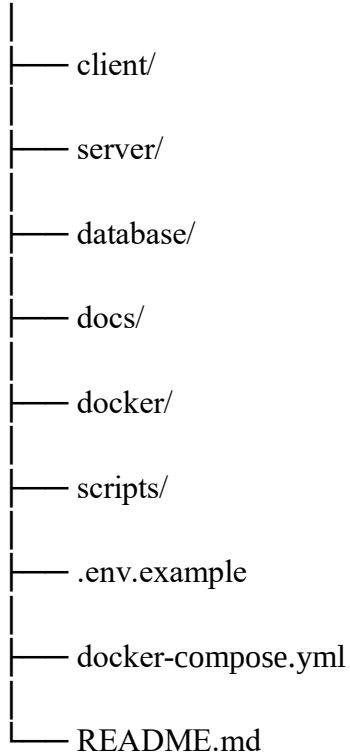
Feature-Based Architecture

Each feature owns:

- Components
 - Hooks
 - Services
 - Types
 - Validation
 - State (if required)
-

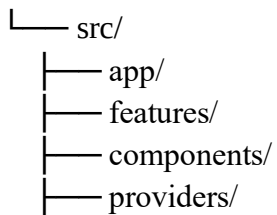
5. Standard Project Structure

project/



Frontend Structure

client/



```
|— hooks/  
|— services/  
|— lib/  
|— contexts/  
|— constants/  
|— styles/  
|— types/  
|— utils/  
|— middleware.ts
```

Backend Structure

```
server/  
└─ src/  
    │— features/  
    │— shared/  
    │— config/  
    │— database/  
    │— common/  
    └─ main.ts
```

Feature Structure

```
auth/  
└─ presentation/  
└─ application/  
└─ domain/  
└─ infrastructure/
```

6. Naming Conventions

Project

- kebab-case

Example:

hospital-management-system

Folders

- kebab-case
-

Files

- kebab-case

Example:

user.service.ts

create-user.dto.ts

jwt-auth.guard.ts

React Components

PascalCase

Example:

UserCard.tsx

LoginForm.tsx

Classes

PascalCase

Interfaces

PascalCase

(No “I” prefix)

Functions

camelCase

Variables

camelCase

Constants

UPPER_SNAKE_CASE

Database

Tables

Plural

Example:

users

products

orders

Columns

snake_case

Example:

created_at

updated_at

phone_number

API

Plural resources

Example

GET /api/v1/users

POST /api/v1/users

DELETE /api/v1/users/{id}

Language Standard

American English

7. Git Workflow

Workflow

GitHub Flow

Branches

- main
- feature/*
- bugfix/*
- hotfix/*
- refactor/*
- docs/*

Commit Standard

Conventional Commits

Examples

feat:

fix:

docs:

refactor:

style:

test:

chore:

8. Frontend Development Standards

Framework

Next.js (App Router)

Language

TypeScript

Styling

Tailwind CSS

Component Library

shadcn/ui

State Management

- Context API
- TanStack Query

Forms

React Hook Form

Validation

Zod

API Client

Axios

Icons

Lucide React

Charts

Recharts

Animations

Framer Motion

Notifications

Sonner

Design Strategy

- Mobile First
 - Responsive
 - Accessible
 - SEO Optimized
-

9. Backend Development Standards

Framework

NestJS

Architecture

Clean Architecture

Business Logic

Use Cases

Data Access

Repository Pattern

Database

Prisma

Validation

class-validator

class-transformer

Logging

Pino

API Documentation

Swagger

Background Jobs

BullMQ

Redis

Configuration

Environment Variables

No hardcoded secrets.

10. Database Standards

Database

PostgreSQL

Primary Keys

UUID

Relationships

Foreign Keys

Migrations

Prisma Migrate

Seed

Prisma Seed

Audit Fields

created_at

updated_at

deleted_at

created_by

updated_by

deleted_by

Soft Deletes

Use deleted_at

11. API Standards

Architecture

REST

Versioning

/api/v1

Format

JSON

Pagination

page

limit

Filtering

Query Parameters

Sorting

sort

order

Search

search

Success Response

```
{  
  "success": true,  
  "message": "Operation completed successfully.",  
  "data": {}  
}
```

Error Response

```
{  
  "success": false,  
  "message": "Validation failed.",  
}
```

```
"errors": []  
}
```

12. Authentication & Authorization

Authentication

JWT

Refresh Tokens

Enabled

Password Hashing

bcrypt

Email Verification

Enabled

Password Reset

Enabled

Authorization

Role-Based Access Control (RBAC)

Permissions

Supported

13. Security Standards

- HTTPS
- Helmet
- CORS
- Rate Limiting
- Input Validation
- SQL Injection Protection (Prisma)
- XSS Protection
- Environment Variables

- Secure Password Hashing
-

14. Testing Standards

Unit Tests

Jest

Integration Tests

Jest

End-to-End Tests

Playwright

API Testing

Postman

Coverage Target

80%+

15. Logging & Monitoring

Logging

Pino

Logs

- Request Logs
- Error Logs
- Security Logs
- Audit Logs

Monitoring

- Sentry
- Uptime Kuma

Health Endpoint

/health

Analytics

Google Analytics

16. File Storage & Email

Images

Cloudinary

Documents

AWS S3

Email

Resend

Email Templates

React Email

Queue

BullMQ

Redis

17. Deployment & DevOps

Frontend

Vercel

Backend

Docker

Reverse Proxy

Nginx

Database

PostgreSQL

Containerization

Docker

Docker Compose

CI/CD

GitHub Actions

Automatic

- Build
 - Test
 - Deploy
-

18. Coding Principles

Every project must follow:

- SOLID
 - DRY
 - KISS
 - YAGNI
 - Separation of Concerns
 - Single Responsibility Principle
 - Dependency Injection
 - Clean Architecture
 - Feature-Based Development
 - Reusable Components
 - Modular Design
 - Secure Coding Practices
-

19. Project Checklist

Before Development

- Requirements gathered

- Database designed
- API documented
- Architecture approved

During Development

- Follow naming conventions
- Use Clean Architecture
- Write reusable components
- Validate all input
- Document APIs

Before Deployment

- Tests passing
- Linting successful
- Build successful
- Documentation updated
- Environment variables configured
- Security reviewed
- Performance reviewed

Our Development Workflow

Every feature follows this process:

Requirement



Documentation



Design



Review



Development



Testing



Documentation Update



Release

Conclusion

This Engineering Handbook serves as the official development standard for all current and future projects.

Every application developed by the company must follow these standards to ensure consistency, scalability, maintainability, security, and long-term success.

This document should be version-controlled and updated whenever engineering practices evolve.